

Practical No. 6

Aim: Implementation of different types of functions with suitable examples.

- **Number Function**
- **Aggregate Function**
- **Character Function**
- **Conversion Function**
- **Date Function**

Theory:

NUMBER FUNCTION:

SQL Numeric Functions are used to perform operations on numeric data types such as **INT**, **FLOAT**, **DECIMAL**, **DOUBLE**, and others. These functions help you manipulate numbers in various ways, including performing **arithmetic operations**, rounding, formatting, and aggregating data.

Commonly Used SQL Numeric Functions

1. ABS() – Absolute Value

The **ABS()** function returns the absolute value of a number, which is the number without its sign (i.e., it converts negative numbers to positive).

Syntax:

```
SELECT ABS(number);
```

Example:

Abs(n): Select abs(-15) from dual;

2. EXP() – Exponential Function

The **EXP()** function returns the value of e raised to the power of a specified number, where e is the base of the natural logarithm (approximately 2.71828).

Syntax:

```
SELECT EXP(number);
```

Example:

Exp(n): Select exp(4) from dual;

3. POWER() – Raise a Number to the Power of Another

The **POWER()** function is used to raise a number to the power of another number. It is often used in mathematical calculations like compound interest or growth rate.

Syntax:

```
SELECT POWER(base, exponent);
```

Example

Power(m,n): Select power(4,2) from dual;

4. MOD() – Modulo or Remainder

The **MOD()** function returns the remainder of a division operation (i.e., it computes the modulus). This function is useful for tasks like determining even/odd numbers or finding remainders in mathematical operations.

Syntax:

```
SELECT MOD(dividend, divisor);
```

Example

Mod(m,n): Select mod(10,3) from dual;

5. ROUND() – Round a Number to a Specified Decimal Place

The **ROUND()** function rounds a number to a specified number of **decimal places**. It is very useful for financial calculations or whenever precise rounding is necessary.

Syntax:

```
SELECT ROUND(number, decimal_places);
```

Example:

Round(m,n): Select round(100.256,2) from dual;

6. TRUNCATE() – Remove Decimal Places

The **TRUNCATE()** function is used to remove the decimal portion of a number without rounding. It truncates the number to the specified number of decimal places.

Syntax:

```
SELECT TRUNCATE(number, decimal_places);
```

Example:

Trunc(m,n): Select trunc(100.256,2) from dual;

7. SQRT() – Square Root

The **SQRT()** function returns the square root of a number. This is useful for mathematical calculations involving geometry or statistical analysis.

Syntax:

```
SELECT SQRT(number);
```

Example:

Sqrt(m,n): Select sqrt(16) from dual;

AGGREGATE FUNCTIONS

SQL Aggregate Functions are essential for summarizing and analyzing large datasets. These functions operate on multiple rows of a table, combining them into a single, more meaningful result. SQL Aggregate functions are mostly used with the GROUP BY clause of the SELECT statement.

Various Aggregate Functions

1. Count()
2. Sum()
3. Avg()
4. Min()
5. Max()

1. Count: COUNT following by a column name returns the count of tuple in that column. If DISTINCT keyword is used then it will return only the count of unique tuple in the column. Otherwise, it will return count of all the tuples (including duplicates) count (*) indicates all the tuples of the column.

Syntax: COUNT (Column name)

Example: SELECT COUNT (Sal) FROM emp;

2. SUM: SUM followed by a column name returns the sum of all the values in that column.

Syntax: SUM (Column name)

Example: SELECT SUM (Sal) From emp;

3. AVG: AVG followed by a column name returns the average value of that column values.

Syntax: AVG (n1, n2...)

Example: Select AVG (10, 15, 30) FROM DUAL;

4. MAX: MAX followed by a column name returns the maximum value of that column.

Syntax: MAX (Column name)

Example: SELECT MAX (Sal) FROM emp;

SQL> select deptno, max(sal) from emp group by deptno;

DEPTNO MAX (SAL)

SQL> select deptno, max (sal) from emp group by deptno having max(sal)<3000;

DEPTNO MAX(SAL)

30 2850

5. MIN: MIN followed by column name returns the minimum value of that column.

Syntax: MIN (Column name)

Example: SELECT MIN (Sal) FROM emp;

SQL>select deptno,min(sal) from emp group by deptno having min(sal)>1000;

DEPTNO MIN (SAL)

10 1300

10 5000

20 3000

30 2850

CHARACTER FUNCTION:

initcap(char) : select initcap("hello") from dual;

lower (char): select lower ('HELLO') from dual;

upper (char) :select upper ('hello') from dual;

ltrim (char,[set]): select ltrim ('cseit', 'cse') from dual;

rtrim (char,[set]): select rtrim ('cseit', 'it') from dual;

replace (char,search): select replace('jack and jue','j','bl') from dual;

CONVERSION FUNCTIONS:

To_char: TO_CHAR (number) converts n to a value of VARCHAR2 data type, using the

optional number format fmt. The value n can be of type NUMBER, BINARY_FLOAT, or BINARY_DOUBLE.

```
SQL>select to_char(65,'RN')from dual;
```

LXV

To_number : TO_NUMBER converts expr to a value of NUMBER data type.

```
SQL>Select to_number ('1234.64') from Dual;
```

1234.64

To_date:TO_DATE converts char of CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to a value of DATE data type.

```
SQL>SELECT TO_DATE('January 15, 1989, 11:00 A.M.')FROM DUAL;
```

TO_DATE

15-JAN-89

STRING FUNCTIONS:

Concat: CONCAT returns char1 concatenated with char2. Both char1 and char2 can be any of the datatypes

```
SQL>SELECT CONCAT('ORACLE','CORPORATION')FROM DUAL;
```

ORACLECORPORATION

Lpad: LPAD returns expr1, left-padded to length n characters with the sequence of characters in expr2.

```
SQL>SELECT LPAD('ORACLE',15,'*')FROM DUAL;
```

*****ORACLE

Rpad: RPAD returns expr1, right-padded to length n characters with expr2, replicated as many times as necessary.

```
SQL>SELECT RPAD ('ORACLE',15,'*') FROM DUAL;
```

ORACLE*****

Ltrim: Returns a character expression after removing leading blanks.

```
SQL>SELECT LTRIM('SSMITHSS','S') FROM DUAL;
```

MITHSS

Rtrim: Returns a character string after truncating all trailing blanks

```
SQL>SELECT RTRIM('SSMITHSS','S') FROM DUAL;
```

SSMITH

Lower: Returns a character expression after converting uppercase character data to lowercase.

```
SQL>SELECT LOWER('DBMS') FROM DUAL;
```

dbms

Upper: Returns a character expression with lowercase character data converted to uppercase

```
SQL>SELECT UPPER('dbms') FROM DUAL;
```

DBMS

Length: Returns the number of characters, rather than the number of bytes, of the given string expression, excluding trailing blanks.

```
SQL>SELECT LENGTH('DATABASE') FROM DUAL;
```

8

Substr: Returns part of a character, binary, text, or image expression.

```
SQL>SELECT SUBSTR('ABCDEFGHIJ',3,4) FROM DUAL;
```

CDEF

19

Instr: The INSTR functions search string for substring. The function returns an integer indicating the position of the character in string that is the first character of this occurrence.

```
SQL>SELECT INSTR('CORPORATE FLOOR','OR',3,2)FROM DUAL;
```

14

DATE FUNCTIONS:

Sysdate:

```
SQL>SELECT SYSDATE FROM DUAL;
```

29-DEC-08

next_day:

```
SQL>SELECT NEXT_DAY(SYSDATE,'WED')FROM DUAL;
```

05-JAN-09

add_months:

```
SQL>SELECT ADD_MONTHS(SYSDATE,2)FROM DUAL;
```

28-FEB-09

last_day:

SQL>SELECT LAST_DAY(SYSDATE)FROM DUAL;

31-DEC-08

months_between:

SQL>SELECT MONTHS_BETWEEN(SYSDATE,HIREDATE)FROM EMP;

4

Least:

SQL>SELECT LEAST('10-JAN-07','12-OCT-07')FROM DUAL;

10-JAN-07

Greatest:

SQL>SELECT GREATEST('10-JAN-07','12-OCT-07')FROM DUAL;

10-JAN-07

Trunc:

SQL>SELECT TRUNC(SYSDATE,'DAY')FROM DUAL;

28-DEC-08

SQL>SELECT ROUND(SYSDATE,'DAY')FROM DUAL;

28-DEC-08

to_char:

SQL> select to_char(sysdate, "dd\\mm\\yy") from dual;

24-mar-05.

to_date:

SQL> select to date (sysdate, "dd\\mm\\yy") from dual;

24-mar-o5.

Note: Write a query to be executed in lab with screen shots.

Result:

Thus, we execute the different types of functions in SQL successfully.